CloudSpartanX

Research Understanding Report

From a broad AI-powered CSPM idea to a focused research direction: DAIL

Item	Current position
Overall product	CloudSpartanX — broader cloud-security product
Current research core	DAIL — Dependency-Aware Invariant Ledger
Current research problem	Regression-aware, stateful Terraform remediation
Phase status	Phase 1 and Phase 2 complete; Phase 3 next
Novelty status	Promising but not yet proven
Patentability status	Not established

This report is a learning/understanding document built from the user-provided Phase 1 and Phase 2 consolidated reports plus primary/authoritative sources where available. It intentionally preserves evidence boundaries and does not turn a provisional gap into a "first ever" claim.

Prepared for research discussion and faculty review

1. Executive Summary

We started with a broad idea: build an AI-assisted cloud security platform that could detect and fix cloud misconfigurations. During Phase 1, we discovered that generic Cloud Security Posture Management (CSPM) is already a mature space. That meant a project whose main contribution was simply "AI that scans cloud security and suggests fixes" would be too close to existing commercial and open-source solutions. Phase 1 therefore narrowed the problem toward AI-generated Infrastructure-as-Code (IaC), especially Terraform, and toward the tension between making an infrastructure change secure and keeping it functionally correct.

Phase 2 was a systematic literature review designed to test that direction rather than assume it was novel. The review confirmed that the basic loop "LLM generates or repairs Terraform → verifier checks it → feedback is returned → LLM tries again" is already increasingly well studied. Systems such as TerraRepair, TerraProbe and verifier-first approaches demonstrate different pieces of this loop. This was a successful research result because it allowed us to reject a weak novelty claim before implementation.

The deeper problem that emerged is regression during iterative repair. An AI may fix one security problem while unintentionally changing another part of the infrastructure that was already correct. The current research direction is therefore DAIL (Dependency-Aware Invariant Ledger): a proposed Terraform-specific mechanism that would keep persistent records of verified properties, understand resource/dependency relationships, and use those records when accepting or rejecting later AI-generated changes.

CloudShield AI has not been replaced by DAIL. CloudShield AI is still the overall product concept; DAIL is the narrower research core inside it.

Phase 1 concluded that generic CSPM novelty was rejected and that the project should move toward a narrow, measurable research problem. The Phase 2 report then refined the problem to preservation of previously verified properties during iterative LLM remediation. filecite markers correspond to the research files used in this project; external references are listed later in this document.

2. The Research Journey in One Story

Step	What happened
1. Broad cloud security	Build an AI-assisted system that detects and fixes cloud security misconfigurations.
2. CSPM reality check	We found that detection, prioritization and AI-assisted remediation are already common in CSPM/CNAPP products.
3. Move to IaC	We narrowed to Infrastructure-as-Code, especially Terraform, because it gives us code, structure, dependencies and reproducible experiments.
4. Initial research loop	LLM proposes Terraform fix → security and functional checks → feedback → another repair.
5. Phase 2 challenge	The literature showed that this basic repair-and-verify loop is already crowded.
6. Deeper failure	Iterative AI repair can damage properties that were already correct, including through unnecessary resource restructuring.
7. New research direction	DAIL: keep a persistent, dependency-aware memory of verified properties and use it during later repair iterations.

3. The Essential Concepts — In Simple Language

Term	Simple meaning
CSPM	Cloud Security Posture Management: a system that continuously looks for risky cloud configurations and helps

Term	Simple meaning
	teams understand and remediate them.
IaC	Infrastructure-as-Code: cloud infrastructure described as code instead of manually configured through a console.
Terraform	A popular IaC tool whose configuration describes the desired cloud state; Terraform plans the changes needed to reach that state.
LLM	A generative AI model that can understand and generate code and natural language.
AI remediation	The AI actually changes the Terraform/IaC configuration to address a security finding; it is not merely recommending a human fix.
Security verification	Checking whether a configuration satisfies security rules or security properties.
Functional verification	Checking whether the infrastructure still behaves as required; in our constrained prototype this can include plan validity, connectivity requirements, tests or sandbox evidence.
Invariant	A property that we want to remain true.
Regression	A property that was correct before a change becomes incorrect after the change.
Deceptive fix	A change that makes a scanner stop reporting a finding without actually removing the underlying security weakness. TerraProbe is specifically aimed at detecting this kind of failure. [2]
Resource identity	The stable identity of a Terraform resource/module that lets the system distinguish the same infrastructure object across changes.
Dependency graph	A map of which infrastructure resources depend on which other resources.
Stateful verification	Verification that carries relevant verified state/properties forward between iterations.
Stateless verification	Each new candidate is checked as a fresh state, without a persistent set of protected historical properties.
Invariant ledger	Our proposed persistent store of verified properties and their relationship to resources/dependencies.
Selective re-verification	Rechecking the properties that a change could affect, plus any required global checks, instead of treating every edit as unrelated.
Formal verification	Using mathematical models and automated reasoning to prove specified properties of a model. It is only as strong as the model and assumptions being verified.

4. Phase 1 — What We Researched and Why

Phase 1 was the problem-discovery and validation stage. Its job was not to design the final system. Its job was to determine whether we were solving a meaningful problem and whether the original idea was differentiated enough for a research project.

Phase 1 component	What it established
Problem landscape	Confirmed that cloud misconfiguration/posture problems are real and important.
Evidence and source verification	Collected evidence, challenged weak statistics, and improved the evidence boundary.
Stakeholder analysis	Compared SOC analysts, DevSecOps/platform engineers, security teams, developers and AI-IaC users.
Root-cause analysis	Identified context gaps, alert/verification problems, remediation friction and AI-generated IaC risk.
Impact analysis	Identified measurable outcomes such as remediation success, regressions, iterations, latency and cost.
Existing-solution landscape	Mapped commercial/open-source/academic approaches and rejected generic CSPM as a sufficient novelty claim.
Candidate selection and feasibility	Compared alternative research directions and constrained the project for a small B.Tech team.
Research charter	Defined the first experimental problem: joint security + functional verification of AI-generated Terraform remediation.

5. Why Generic AI-Powered CSPM Was Rejected

In simple terms, a generic AI-CSPM would look attractive because it combines cloud scanning, risk explanation and AI remediation. The research problem was that these capabilities are already common. A research contribution needs a sharper question than “Can AI find and fix cloud misconfigurations?”

That rejection was a positive result, not a failure. Phase 1 prevented us from spending months building a technically impressive but academically weak product.

- CloudShield can still contain CSPM-style detection in the eventual product.
- The research contribution cannot be generic CSPM detection or generic AI remediation.
- The research focus moved to the behavior of AI-generated Terraform changes and their verification.

The Phase 1 consolidated report explicitly records generic CSPM as saturated/rejected for the intended research contribution and identifies AI-IaC plus verification as the narrower direction. See the Phase 1 consolidated report, sections on what Phase 1 established and final assessment. [Project source: Phase 1 Consolidated Research Report]

6. The Original Phase 1 Research Idea

At the end of Phase 1, the working idea was a loop:

```
LLM generates/repairs Terraform
        ↓
Security verification + functional verification
        ↓
Feedback to the LLM
        ↓
LLM generates a better patch
        ↓
Repeat until accepted or budget exhausted
```

The intended comparison was a security-only verification path versus a joint security + functional verification path, with metrics such as functional regression rate, repair success, number of iterations and verification latency.

Phase 2 was necessary because a good research idea is not “something we can build”; it is something that adds knowledge beyond what has already been demonstrated.

7. Phase 2 — Systematic Literature Review (SLR)

An SLR is a structured academic process for searching, screening and comparing prior studies using a predefined method. We used it to answer a precise question: "Has someone already built and evaluated an AI system that writes or repairs Terraform and verifies the result?"

We used a combination of academic databases and AI-assisted discovery. We deliberately documented the database limitation instead of pretending that a missing search was a zero-result search.

Stage	What we did
Search strategy	Created search families covering LLM + IaC, remediation, formal verification, security invariants, functional correctness, CEGAR/CEGIS and AWS reasoning.
Databases	Worked with IEEE Xplore and ACM; also used the broader planned strategy for Web of Science, ScienceDirect, SpringerLink, Google Scholar/Semantic Scholar. Scopus was inaccessible during collection and is recorded as a limitation.
AI discovery	Used Elicit, Gemini Deep Research and related literature discovery workflows to expand the candidate set.
Citation expansion	Used seed-paper relationships and citation discovery; ResearchRabbit/Scite were part of the intended workflow, while the final evidence depended on the records actually verified.
Organization	Zotero was treated as the master reference library.
Screening	Title/abstract screening followed by full-text verification when primary text was available.
Extraction	Compared studies across 21 fields, emphasizing stateful verification, invariant state, resource identity, dependency tracking and property caching.
Synthesis	Grouped the literature into themes, contradictions, mature areas, underexplored areas and prior-art threats.

8. What the Major Systems Taught Us

Study	What it already demonstrates	Why it matters to DAIL
TerraRepair	Tool-grounded LLM agent for Terraform repair. Uses dependency context, provider schema context and scanner reruns. It strongly demonstrates the LLM + Terraform + dependency + scanner loop. It does not demonstrate the persistent cross-iteration invariant ledger proposed by DAIL. [1]	Direct prior art for the old idea; not the complete DAIL mechanism.
TerraProbe	Layered evaluation framework for LLM-assisted Terraform security repair. It checks more than targeted scanner disappearance, including planning/semantic layers. It is primarily an evaluation/oracle framework rather than a persistent repair-state controller. [2]	Shows why simple scanner-passing is not enough and motivates stronger verification.
SafeFix-Agent	The supplied Phase 2 material describes policy, test and sandbox gates around LLM remediation. We could not independently verify all implementation/results from a primary source during this report, so this item is treated cautiously.	Useful adjacent evidence, but not a basis for a strong novelty claim.
Verifier-First	Uses Terraform validate, terraform plan	Confirms that verifier-feedback loops

Study	What it already demonstrates	Why it matters to DAIL
	and OPA policy evaluation in a verifier-first agentic generation workflow. [3]	already exist; differs from DAIL in its lack of a Terraform-specific persistent invariant ledger.
LLMSecConfig	LLM + RAG + static analysis for Kubernetes security misconfiguration repair; reports operational-function preservation. [4]	Strong adjacent-domain evidence: repeated checking exists, but it is Kubernetes rather than Terraform and is not a DAIL-style ledger.
ConVer	Uses LLM-generated contracts and CEGAR/CEGIS-style refinement for formal software verification. [5]	Strongest methodological adjacent threat. It shows that persistent contract/invariant refinement is not itself new, but its domain is C programs, not Terraform cloud resources.
PolyVer	Uses compositional verification and LLM/synthesis-generated contracts for polyglot systems, with CEGAR/CEGIS. [6]	Shows contract-based and compositional formal verification in another domain.
Zelkova	AWS formal reasoning engine for IAM/access-policy properties using SMT. [7]	Strong foundation for formal cloud security reasoning; not an LLM Terraform repair loop.
Tiros	AWS network reachability reasoning using automated theorem proving. [8]	Foundation for network/functionality-related cloud verification; not an iterative LLM repair system.
IAMPERE / UPIR	Kept provisional in our review because the Phase 2 evidence set did not consistently provide sufficiently verified primary-source material for strong claims.	Do not use as decisive proof until independently verified in Phase 3/continued review.

9. The Biggest Discovery of Phase 2

The most important finding was that the basic repair loop is no longer a sufficient research contribution. The literature contains systems that already ground an LLM in Terraform/provider context, run security scanners, validate Terraform, apply policy checks, use layered oracles, and perform iterative refinement. [1–4]

The more interesting failure is what happens over time. A later repair can fix a new issue but accidentally destroy a property that was already correct. Recent empirical work directly studies this phenomenon: the 2026 “Does Fixing Break Security?” study analyzed 5,968 IaC-Eval scenario timelines and reported regression in 13.8% of scenarios under standard detection and 3.3% under a stricter definition; it identified resource restructuring as the dominant reported root cause at 79.0%. [9]

This does not mean every apparent regression is a real security regression; the stricter rate is important because many apparent regressions can be measurement artifacts. That is precisely why our eventual experiment must define regression carefully.

10. A Simple Example of the Regression Problem

Imagine a web application running on EC2. The web server has a security group. The application also needs a network path to a database. The AI is asked to fix an overly permissive inbound rule.

```
Before:
Web Server → Security Group → required application path → Database
                  ↑
           security issue to fix
```

The AI produces a patch. It successfully closes the risky rule, so the security scanner is happy. But suppose the AI rewrites the whole security-group block and accidentally removes or changes the rule needed by the application to reach the database.

```
After:
Web Server → Security Group  X → required application path → Database
                  ↑
          scanner may show “fixed”
          application may be broken
```

The security goal improved, but a previously correct functional property was broken. That is the core kind of regression we want to study. The exact functional invariant in a future prototype must be defined narrowly and explicitly; we are not claiming that every real-world application can be represented by a universal invariant.

11. Stateless vs. Stateful Verification

Stateless approach	Stateful approach (DAIL candidate)
Check the current candidate as a fresh state.	Carry forward a verified set of important properties.
A scanner can tell us what fails now.	The system also knows what it previously accepted as protected.
Past success may be rechecked, but is not represented as an explicit protected state.	Previous invariants become acceptance constraints for later iterations.
Usually whole-file or whole-policy rechecking.	Dependency information can help decide what needs re-verification.
Simple to implement.	More complex; must be tested for false rejection and soundness.

Important: simply rerunning Checkov, Trivy or another scanner is not automatically the same as a persistent invariant ledger. A rerun asks, "What does the current configuration violate?" A ledger-based design asks, "What did we previously establish as protected, and has this new change invalidated it?" The two mechanisms can be combined, but they answer different research questions.

12. What Is an Invariant Ledger?

An invariant is a property we want to remain true. Examples in a constrained Terraform experiment might be: "Production EC2 must not be publicly exposed on SSH" or "Application EC2 must retain required database connectivity on port 5432." The ledger is the persistent research structure that records which properties have been verified, what resources/dependencies they are attached to, and what conditions would require them to be checked again.

```
Ledger example (conceptual only)

Invariant I1: Prod EC2 has no public SSH exposure       = PASS
Invariant I2: App EC2 can reach DB on port 5432        = PASS
Invariant I3: DB is not publicly reachable             = PASS

Protected resources/dependencies:
I1 → SG-Web, EC2-Web
I2 → EC2-App, SG-App, EC2-DB, network path
I3 → EC2-DB, SG-DB
```

This is a conceptual model, not the final data structure. Phase 3 must determine how invariants are represented, stored, linked to resources and invalidated.

13. What Does Dependency-Aware Mean?

Terraform infrastructure is not just a flat list of files. Resources can reference one another. A useful mental model is a graph:

```
EC2 Web
   │
   ▼
Security Group
   │
   ▼
Network / subnet relationships
   │
   ▼
Database
```

If the AI changes the Security Group, a dependency-aware system should reason about which previously verified properties may be affected. This motivates selective re-verification. But dependency context alone is not enough: TerraRepair already uses dependency context. DAIL would need a rule that turns dependency relationships into a preservation/re-verification mechanism.

14. CloudShield AI vs. DAIL — The Architecture

CloudShield AI is the overall product. DAIL is the research engine inside it. This separation is important: the product can contain detection, explanations, reporting and automation even though the academic contribution is much narrower.

```
CloudShield AI (Overall Product)
│
├── 1. Detection / CSPM
│      Find cloud/IaC security issues
│
├── 2. Context & Risk
│      Explain affected resources, dependencies and severity
│
├── 3. AI Remediation
│      Generate candidate Terraform patches
│
├── 4. DAIL Research Engine
│      ├── Invariant State
│      ├── Resource Identity
│      ├── Dependency Analysis
│      └── Regression Control
│
└── 5. Verification & Reporting
       Show accepted/rejected fixes and evidence
```

This architecture is conceptual. The exact internal data structures, invariant language, whether SMT/Z3 is required, and the final selective re-verification algorithm are intentionally not locked yet.

15. DAIL — Candidate Workflow

- 1. Capture a baseline Terraform configuration and its resource/dependency structure.
- 2. Detect or receive a security finding.
- 3. Ask an LLM to generate a constrained Terraform patch.
- 4. Run the required current-state security/functional verification.
- 5. Compare the candidate against the protected invariant state.
- 6. Determine whether protected resources or dependencies were unintentionally changed.
- 7. Re-verify affected properties and any required global invariants.
- 8. Reject the patch if protected properties fail; otherwise accept and update the ledger.
- 9. Continue for a bounded number of iterations, collecting repair/regression metrics.

16. Current Research Question

RQ1: How can persistent, dependency-aware invariant preservation be used to reduce unintended security and infrastructure regressions during iterative LLM-based Terraform remediation?

In plain English: Can we give the AI a reliable memory of the infrastructure properties that were already proven correct, understand what those properties depend on, and use that memory to stop later AI fixes from accidentally breaking working infrastructure?

17. Current Secondary Questions and Hypotheses

Question	Current wording
RQ2	Does explicit invariant state reduce regression rates compared with stateless verification?
RQ3	Does resource-identity-aware acceptance reduce unrelated Terraform restructuring?
RQ4	Can dependency-aware selective re-verification preserve correctness while reducing unnecessary verification work?
RQ5	What trade-offs does invariant preservation introduce in repair success, iteration count, latency and implementation complexity?
Hypothesis	**Statement**
H1	Stateful invariant preservation will reduce unintended regressions relative to stateless verification.
H2	Resource-identity-aware acceptance will reduce unrelated resource mutations during iterative repair.
H3	Dependency-aware selective re-verification can preserve protected properties while reducing unnecessary re-checks.
H4	The added state/verification machinery will remain computationally feasible for a constrained Terraform subset.

18. What DAIL Is — and Is Not

DAIL is	DAIL is not
A candidate Terraform-specific regression-control mechanism.	A claim that LLM + Terraform + verification is novel.
A persistent representation of verified properties.	A new general formal-verification paradigm.
Dependency- and resource-aware acceptance logic.	Merely a scanner wrapper.
A mechanism to be tested experimentally.	Just prompt engineering.
A possible way to preserve previously verified state.	Necessarily an SMT/Z3-only system.
A research core inside CloudShield AI.	A claim of complete cloud-security correctness.

19. What Is Established vs. What Is Still Open

Status	What belongs here
Established	Generic CSPM is too saturated to be our primary academic contribution; AI-assisted Terraform remediation and verifier-feedback loops already exist; scanner-only success can be misleading; the product/research split is CloudShield AI/DAIL.
Strongly supported	Iterative AI remediation can introduce regressions; resource restructuring is an important reported failure mode; formal methods provide useful adjacent mechanisms for invariants/contracts.
Provisional	DAIL as the best Terraform-specific research-gap candidate;

Status	What belongs here
	the exact invariant-ledger design; resource-identity preservation as a key mechanism.
Not yet established	That DAIL is the first such system; that it is patentable; that it will outperform a stateless baseline; that Z3/SMT is necessary; that it will scale to enterprise infrastructure.

The Phase 2 consolidated report explicitly states that DAIL is a leading research-gap candidate, not a proven novel invention, and that the exact ledger structure, invariant language, Z3/SMT requirement, selective re-verification algorithm and benchmark remain deliberately open. [Project source: Phase 2 Consolidated SLR]

20. Evidence and "Proof" for Key Claims

The following sources are the strongest evidence used to support statements in this report. Where a source was not independently verified during this phase, the report says so rather than treating the claim as established.

Source	What it supports	Link
[1] TerraRepair (2026)	Primary arXiv record. Confirms tool-grounded Terraform repair, dependency context, provider schema use, scanner reruns, benchmark scope and reported fix-rate results.	https://arxiv.org/abs/2607.11390
[2] TerraProbe (2026)	Primary arXiv record. Confirms the layered-oracle Terraform security-repair evaluation focus.	https://arxiv.org/abs/2606.26590
[3] Verifier-First (2026)	Primary arXiv record. Confirms Terraform validate/plan/OPA stages and iterative verifier feedback on IaC-Eval v2.	https://arxiv.org/abs/2607.20478
[4] LLMSecConfig (2025)	Primary arXiv record. Confirms LLM + RAG + static analysis for Kubernetes misconfiguration repair and the reported evaluation.	https://arxiv.org/abs/2502.02009
[5] ConVer (2026)	Primary arXiv record. Confirms LLM-generated contracts and CEGAR/CEGIS-style formal verification for C programs.	https://arxiv.org/abs/2605.27051
[6] PolyVer (2025)	Primary arXiv record. Confirms compositional verification using contracts, LLM/synthesis oracles, and CEGAR/CEGIS for polyglot C/Rust systems.	https://arxiv.org/abs/2503.03207
[7] Zelkova (2018)	Amazon Science/FMCAD record. Confirms SMT-based reasoning over AWS access policies.	https://www.amazon.science/publications/semantic-based-automated-reasoning-for-aws-access-policies-using-smt
[8] Tiros (2019)	Chalmers/CAV record and full paper. Confirms automated reasoning for AWS network reachability and security invariants.	https://research.chalmers.se/en/publication/511601
[9] Does Fixing Break Security? (2026)	Primary arXiv record and ESEIW/ESEM conference page. Confirms the empirical study of per-iteration security regression in LLM-driven IaC repair and the reported regression statistics.	https://arxiv.org/abs/2608.13404
[10] Security Invariants (SI), AWS/ASE 2024)	Amazon Science/ASE record. Useful evidence that formal	https://www.amazon.science/publications/cloud-resource-protection-via-automated-security-

Source	What it supports	Link
	security invariants and cross-resource cloud reasoning are already industrial research topics.	property-reasoning

Evidence note: SafeFix-Agent was retained as a relevant study from the project's Phase 2 material, but I could not independently retrieve a reliable primary source in this verification pass. I therefore do not use it as decisive proof of novelty. IAMPERE and UPIR are likewise treated cautiously because our project evidence set did not consistently contain sufficiently verified primary-source material for strong claims about their exact overlap with DAIL. The primary UPIR disclosure does exist, but it is a technical disclosure rather than a peer-reviewed Terraform study; it is best treated as adjacent formal-method inspiration rather than direct Terraform prior art. https://www.tdcommons.org/dpubs_series/8852/

21. Important Limitations and Evidence Boundaries

- Scopus was inaccessible during the collection stage; we recorded this as a limitation rather than fabricating a result count.
- Not every planned database search has a validated execution log.
- Some early screening and extraction decisions relied on abstracts or AI-generated summaries; those are not equivalent to full-paper review.
- Publication status varies across peer-reviewed work, preprints and industry/vendor material.
- The exact DAIL mechanism is not yet designed; therefore the research gap cannot be converted into a final novelty claim from the architecture sketch alone.
- The invariant ledger may not be necessary if a simpler final-state verification system detects every relevant regression cheaply; this is an empirical question.
- A poorly designed ledger could create false rejection or lock-in, so Phase 3 must test soundness and failure modes.
- Formal verification only proves properties of the selected model and invariants; it does not prove complete cloud security in the real world.

22. What Is Still Left Before Implementation

Phase 3 is not simply "start coding." It is the formalization stage that converts the current research candidate into an exact, falsifiable system design.

- Precisely define the Terraform subset and resource model.
- Define the invariant representation and what counts as "verified."
- Define stable resource identity and how identity changes are handled.
- Define the dependency graph and invalidation rules.
- Define the repair-acceptance algorithm.
- Define the stateless baseline and any stronger baselines.
- Define benchmark scenarios and datasets.
- Define regression, repair-success, iteration, latency and cost metrics.
- Define failure/soundness boundaries and what the system cannot prove.
- Perform another targeted prior-art/patent check against the exact mechanism once specified.

23. The Faculty-Level Explanation

30-second version:

"We started with AI-based cloud security, but generic CSPM was already crowded. We narrowed to AI-generated Terraform security remediation. The literature showed that basic LLM repair-and-verify loops already exist. Our research focuses on the deeper problem that an AI can fix one issue and break another property that was previously correct. DAIL is our candidate mechanism for remembering verified properties and using Terraform dependencies/resource identity to prevent those regressions. CloudShield AI remains the overall product."

1-minute version:

“The product is CloudShield AI, a broader cloud-security platform. The research contribution is DAIL, a Terraform-focused verification mechanism. Instead of treating every AI-generated patch as a fresh state, DAIL would maintain a persistent set of verified security/functional properties, map them to resource/dependency identity, and reject later patches that break protected properties. We still need to formalize and experimentally test the mechanism; we are not claiming novelty or patentability yet.”

24. Final Conclusion of Phase 1 + Phase 2

The two phases successfully changed the project from a broad product idea into a narrow research problem. The biggest success is not that we already have a finished algorithm. The biggest success is that we used evidence to reject weak contributions before implementation.

The project is now at a clear decision point: CloudShield AI remains the broad product vision, while DAIL is the current research core. The SLR did not prove DAIL to be the first or only system of its kind. It did show a fragmented landscape in which Terraform-specific repair systems provide tool grounding and layered verification, while adjacent formal-methods systems provide invariant/contract refinement. The exact combination we want to test — persistent verified property state, resource/dependency identity, and regression-aware selective re-verification during iterative LLM Terraform remediation — remains a strong and testable research-gap candidate in the evidence reviewed so far.

Phase 3 should now formalize the mechanism, define what it can and cannot prove, and design a controlled experiment against stateless baselines. Only after that should we make stronger claims about novelty, contribution or patentability.

25. References

[1] TerraRepair: A Tool-Grounded LLM Agent for Infrastructure-as-Code Repair — https://arxiv.org/abs/2607.11390

[2] TerraProbe: A Layered-Oracle Framework for Detecting Deceptive Fixes in LLM-Assisted Terraform Security Repair — https://arxiv.org/abs/2606.26590

[3] Verifier-First Evaluation of Agentic LLMs for Infrastructure-as-Code Generation — https://arxiv.org/abs/2607.20478

[4] LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations — https://arxiv.org/abs/2502.02009

[5] ConVer: Using Contracts and Loop Invariant Synthesis for Scalable Formal Software Verification — https://arxiv.org/abs/2605.27051

[6] PolyVer: A Compositional Approach for Polyglot System Modeling and Verification — https://arxiv.org/abs/2503.03207

[7] Semantic-based automated reasoning for AWS access policies using SMT (Zelkova), FMCAD 2018 — https://www.amazon.science/publications/semantic-based-automated-reasoning-for-aws-access-policies-using-smt

[8] Reachability analysis for AWS-based networks (Tiros), CAV 2019 — https://research.chalmers.se/en/publication/511601

[9] Does Fixing Break Security? An Empirical Study of Security Degradation in Iterative LLM-Driven Infrastructure-as-Code Repair — https://arxiv.org/abs/2608.13404

[10] Cloud resource protection via automated security property reasoning (Security Invariants), ASE 2024 — https://www.amazon.science/publications/cloud-resource-protection-via-automated-security-property-reasoning

[11] Automated Synthesis and Verification of Distributed Systems Using UPIR, Technical Disclosure Commons, 2025 — https://www.tdcommons.org/dpubs_series/8852

Project-source references: CloudShield_AI_Phase_1_Consolidated_Research_Report_v1.0.docx; CloudShield_AI_Phase_2_Consolidated_SLR_Report_v1.0.docx; CloudShield_AI_Phase_1_2_Faculty_Progress_and_Research_Direction_Report.docx.